

Условие:

Разрядность адреса – 52; размер кэш-памяти – 1 Мбайт; размер блока – 128 Б; ассоциативность – 8; тип записи – отложенная (write-back); политика заведения – промах по чтению; политика вытеснения – LRR (Less recently read).

Структура адреса:

35 бит – тег (остаток в тег)
10 бит – индекс (число наборов = $1 \text{ Мбайт} / 128 / 8 = 1024$)
7 бит – сдвиг (смещение в блоке)

Для упрощения рассмотрения тег искусственно обрезается до 4 бит

Конечный автомат Мили:

Состояние автомата:

8 наборов по:

```
{
    Бит валидности, (Val)
    Бит модифицированности, (Mod)
    3 бита времени жизни, ([2:0] TOL)
    4 бита тега ([3:0] Tag)
}
```

S – все комбинации состояний автомата

X – все комбинации наборов:

```
{
    2 бита кодировка операции (Op: {Write, Read, Snoop, Noop})
    4 бит тега
}
```

Y:

```
{
    1 бит валидности (Val)
    1 бит записи в память (Wb)
}
```

S0 : все нули

Функции выходов:

```
{
    Exist (tag) == {в одном из наборов в состоянии Val == 1 && Tag == tag}
}
```

If X.Op == Snoop && Exist (X.tag) && State[N].Mod == 1

N - номер набора, где нашелся Exist

Y = {1, 1}

If X.Op == Read && not Exist (X.tag) && not Has_Invalid &&
State[N].Mod == 1

N - номер набора, с наибольшим TOL (TOL == 7)

Y = {1, 1}

If X.Op == Write && not Exist (X.tag)

Y = {0, 1}

If X.Op == Read, && Exist (X.tag)

N - номер набора, где нашелся Exist

Y = {1, 0}

Else Y = {0, 0}

}

Функции переходов:

{

Has_Invalid == {в одном из наборов в состоянии Val == 0}

New_State = State

If X.Op == Snoop && Exist (X.tag)

N - номер набора, где нашелся Exist

New_State[N].Valid = 0

New_State[N].Mod = 0

New_State[N].Tag = 0

New_State[N].TOL = 7

If X.Op == Write && Exist (X.tag)

N - номер набора, где нашелся Exist

New_State[N].Mod = 1

If X.Op == Read && Exist (X.tag)

N - номер набора, где нашелся Exist

New_State[N].TOL = 0

Other – пробегает остальные наборы, кроме N

If New_State[other].TOL < State[N].TOL

New_State[other].TOL += 1

If X.Op == Read && not Exist (X.tag) && Has_Invalid

N - номер набора, где нашелся Val == 0

New_State[N].TOL = 0

Other – пробегает остальные наборы, кроме N

If New_State[other].TOL < State[N].TOL

```

        New_State[other].TOL += 1
        New_State[N].Mod = 0
        New_State[N].Val = 1
        New_State[N].Tag = X.Tag

    If X.Op == Read && not Exist (X.tag) && not Has_Invalid
        N - номер набора, с наибольшим TOL (TOL == 7)
        New_State[N].TOL = 0
        Other – пробегает остальные наборы, кроме N
        If New_State[other].TOL < State[N]. TOL
            New_State[other].TOL += 1
            New_State[N].Mod = 0
            New_State[N].Val = 1
            New_State[N].Tag = X.Tag
    }

```

Минимальная запись в память: (заведение в кеш = промах по чтению)
 ЗапросN = {Операция, {адрес памяти}}

Запрос0 = { Write, {2'10001000100010111}} // Offset = 7, Addr = 2'1000100010, Tag = 2'1000

S0 = {{0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}}
 Y = {0, 1}

Минимальное вытеснение или вычёркивание значимого блока из набора:

```

Запрос0 = { Read, {2'10001000100010111}}
Запрос1 = { Read, {2'10011000100010111}}
Запрос2 = { Read, {2'10101000100010111}}
Запрос3 = { Read, {2'10111000100010111}}
Запрос4 = { Read, {2'11001000100010111}}
Запрос5 = { Read, {2'11011000100010111}}
Запрос6 = { Read, {2'11101000100010111}}
Запрос7 = { Read, {2'11111000100010111}}
Запрос8 = { Read, {2'00011000100010111}}

```

Ss -> Sf: {Sf} / {Y}

S0 -> S1: {{0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {1, 0, 0, 2'1000}} / {0, 0}

S1 -> S2: {{0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {0, 0, 7, 0}, {1, 0, 0, 2'1001}, {1, 0, 1, 2'1000}} / {0, 0}

S2 -> S3: $\{\{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{1, 0, 7, 2'1010\}, \{1, 0, 1, 2'1001\}, \{1, 0, 2, 2'1000\}\} / \{0, 0\}$

S3 -> S4: $\{\{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{1, 0, 0, 2'1011\}, \{1, 0, 1, 2'1010\}, \{1, 0, 2, 2'1001\}, \{1, 0, 3, 2'1000\}\} / \{0, 0\}$

S4 -> S4: $\{\{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{1, 0, 0, 2'1100\}, \{1, 0, 1, 2'1011\}, \{1, 0, 2, 2'1010\}, \{1, 0, 3, 2'1001\}, \{1, 0, 4, 2'1000\}\} / \{0, 0\}$

S5 -> S6: $\{\{0, 0, 7, 0\}, \{0, 0, 7, 0\}, \{1, 0, 0, 2'1101\}, \{1, 0, 1, 2'1100\}, \{1, 0, 2, 2'1011\}, \{1, 0, 3, 2'1010\}, \{1, 0, 4, 2'1001\}, \{1, 0, 5, 2'1000\}\} / \{0, 0\}$

S6 -> S7: $\{\{0, 0, 7, 0\}, \{1, 0, 0, 2'1110\}, \{1, 0, 1, 2'1101\}, \{1, 0, 2, 2'1100\}, \{1, 0, 3, 2'1011\}, \{1, 0, 4, 2'1010\}, \{1, 0, 5, 2'1001\}, \{1, 0, 6, 2'1000\}\} / \{0, 0\}$

S7 -> S8: $\{\{1, 0, 0, 2'1111\}, \{1, 0, 1, 2'1110\}, \{1, 0, 2, 2'1101\}, \{1, 0, 3, 2'1100\}, \{1, 0, 4, 2'1011\}, \{1, 0, 5, 2'1010\}, \{1, 0, 6, 2'1001\}, \{1, 0, 7, 2'1000\}\} / \{0, 0\}$

S8 -> S9: $\{\{1, 0, 1, 2'1111\}, \{1, 0, 2, 2'1110\}, \{1, 0, 3, 2'1101\}, \{1, 0, 4, 2'1100\}, \{1, 0, 5, 2'1011\}, \{1, 0, 6, 2'1010\}, \{1, 0, 7, 2'1001\}, \{1, 0, 0, 2'0001\}\} / \{0, 0\}$

Достижимость состояний:

бит Val каждого набора из состояния может быть 1 или 0 вне зависимости от остальных значений набора

бит Mod каждого набора из состояния может быть 1 или 0 только при условии, что бит Val этого набора = 1

биты Тега каждого набора из состояния могут быть 1 или 0 вне зависимости от остальных значений, кроме случая совпадения с другой ячейкой

биты TOL каждого набора зависят от состояний других наборов и могут либо

- быть 7, когда бит валидности 0, но независимо от других наборов

- когда бит валидности 1, быть упорядочены от 0 до 7 среди всех наборов

Количество состояний:

$2*6!$ – 6 битов Val = 1, Mod = 0/1, TOL = [0-5]

$2*(8!+7!+6!+5!+4!+3!+2!+1!) + 1$ – модифицированность, TOL и валидность для 8 наборов

$16!/(16!-8)$ – теги

Итого: $92467 * 4151347200 \sim 3e+14$

Пройдено: 9

Процент покрытия КА: $3e-12\%$

Количество переходов: Количество состояний * Размер входных данных =
 $3e+14 * 2^{(2 + 4)} = 3e+14 * 64$

Пройдено: 10

Процент покрытия КА: $\sim 5e-14\%$